

CS2601 Linear and Convex Optimization

Homework 7 Solution

Due: 2021.11.20

For this assignment, you should submit a **single** pdf file as well as your source code (.py or .ipynb files). The pdf file should include all necessary figures, the outputs of your Python code, and your answers to the questions. Do NOT submit your figures in separate files. Your answers in any of the .py or .ipynb files will NOT be graded.

In this problem, you will implement gradient descent with backtracking line search. First complete the function `gd_armijo` in `gd.py`.

1. Consider the following optimization problem in (9.20) of Boyd and Vandenberghe,

$$\min_{x_1, x_2} f(x_1, x_2) = e^{x_1+3x_2-0.1} + e^{x_1-3x_2-0.1} + e^{-x_1-0.1} \quad (1)$$

- (a). What's the optimal solution $\mathbf{x}^*$ and the optimal value $f(\mathbf{x}^*)$?
- (b). Solve (1) numerically using your implementation of gradient descent with backtracking line search. Use $\alpha = 0.1$ and $\beta = 0.7$ as in Boyd and Vandenberghe. Use the initial points $\mathbf{x}_0 = (-1.5, 1)^T$. Report the solution, the number of iterations in the outer loop and the total number of iterations in the inner loop. Plot the trajectory of $\mathbf{x}_k$, the error $f(\mathbf{x}_k) - f(\mathbf{x}^*)$ and the step sizes t_k .
- (c). Solve (1) using your implementation of gradient descent with constant step size in HW6. Use $\mathbf{x}_0 = (-1.5, 1)^T$. Report the solution and the number of iterations for step sizes 0.1 and 0.01.
- (d). Redo part (b) using the initial point $\mathbf{x}_0 = (1.5, 1)^T$.
- (e). Redo part (c) using $\mathbf{x}_0 = (1.5, 1)^T$ and step size 0.005. Report the solution and the number of iterations. What happens if you use the step sizes in part (c)?

Solution. (a). f is convex, since exponential functions are convex, and composition with affine functions and nonnegative combination preserve convexity. The optimal solution satisfies $\nabla f(\mathbf{x}^*) = \mathbf{0}$.

The gradient is

$$\nabla f(\mathbf{x}) = \begin{pmatrix} e^{x_1+3x_2-0.1} + e^{x_1-3x_2-0.1} - e^{-x_1-0.1} \\ 3e^{x_1+3x_2-0.1} - 3e^{x_1-3x_2-0.1} \end{pmatrix}$$

Setting $\nabla f(\mathbf{x}) = \mathbf{0}$, we obtain the optimal solution $\mathbf{x}^* = (-\frac{1}{2} \log 2, 0)^T$ and $f^* = f(\mathbf{x}^*) = 2\sqrt{2}e^{-0.1}$.

- (b). `gradient descent with Armijo`

number of iterations in outer loop: 28
 total number of iterations in inner loop: 151
 solution: [-3.46574284e-01 3.04072749e-07]
 value: 2.5592666966593645

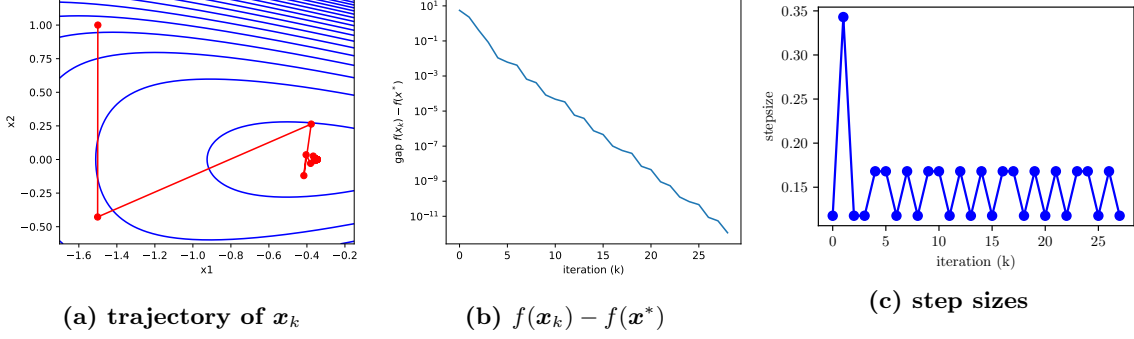


Figure 1: $x_0 = (-1.5, 1)^T$

(c). gradient descent with constant stepsize 0.1
 number of iterations: 44
 solution: [-3.46576607e-01 3.21465960e-18]
 value: 2.559266696669859

gradient descent with constant stepsize 0.01
 number of iterations: 489
 solution: [-3.46577419e-01 8.65140907e-18]
 value: 2.559266696676969

(d). gradient descent with Armijo
 number of iterations in outer loop: 32
 total number of iterations in inner loop: 197
 solution: [-3.4657238e-01 6.5447655e-07]
 value: 2.5592666966625575

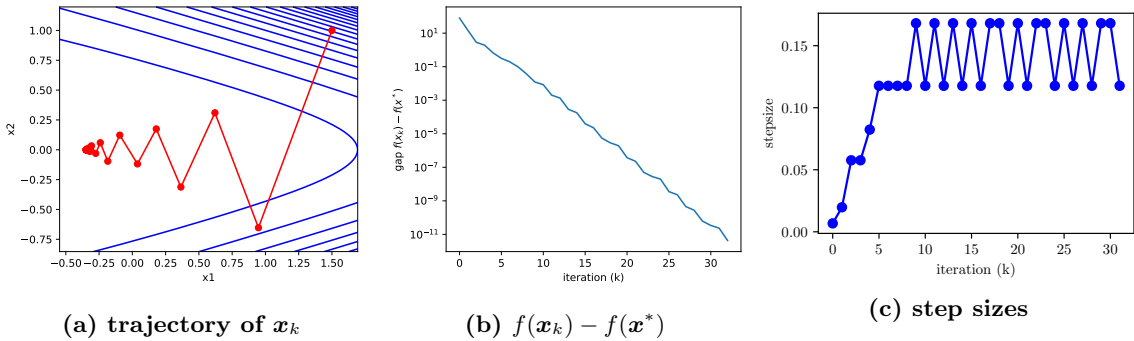


Figure 2: $x_0 = (1.5, 1)^T$

(e). gradient descent with constant stepsize 0.005

```

number of iterations: 984
solution: [-3.46569713e-01 -8.97551794e-18]
value: 2.559266696677449

```

Gradient descent diverges if we use step size 0.01.

□

A possible implementation.

```

def gd_armijo(f, fp, x0, initial_stepsize=1.0, alpha=0.5, beta=0.5, tol=1e-5, maxiter=100000):
    x_traces = [np.array(x0)]
    stepsize_traces = []
    tot_num_inner_iter = 0

    x = np.array(x0)

    for it in range(maxiter):
        grad = fp(x)

        if np.linalg.norm(grad) < tol:
            break

        stepsize = initial_stepsize
        fx = f(x)
        alpha_grad_squared = alpha * (grad @ grad)

        while f(x - stepsize * grad) > fx - stepsize * alpha_grad_squared:
            stepsize *= beta
            tot_num_inner_iter += 1

        x -= stepsize * grad
        x_traces.append(np.array(x))
        stepsize_traces.append(stepsize)

    return x_traces, stepsize_traces, tot_num_inner_iter

```

2. Noisy gradient. Consider an m -strongly convex and L -smooth function f with a global minimum $\mathbf{x}^*$. Suppose we try to find $\mathbf{x}^*$ using gradient descent, but we can only get an approximation to the true gradient in each iteration. More precisely, the approximate gradient we get in the k -th iteration is $\mathbf{g}_k = \nabla f(\mathbf{x}_k) + \boldsymbol{\varepsilon}_k$, where $\boldsymbol{\varepsilon}_k$ is some noise term with norm bounded by E , i.e. $\|\boldsymbol{\varepsilon}_k\| \leq E$ for all k . Suppose we use constant step size $t \in (0, 1/L]$. Then the gradient step becomes,

$$\mathbf{x}_{k+1} = \mathbf{x}_k - t\mathbf{g}_k = \mathbf{x}_k - t(\nabla f(\mathbf{x}_k) + \boldsymbol{\varepsilon}_k).$$

(a). Let $\tilde{\mathbf{x}}_{k+1} = \mathbf{x}_k - t\nabla f(\mathbf{x}_k)$, which is the iterate we would get in the k -th iteration if $\boldsymbol{\varepsilon}_k = \mathbf{0}$. Show

$$\|\mathbf{x}_{k+1} - \mathbf{x}^*\| \leq \|\tilde{\mathbf{x}}_{k+1} - \mathbf{x}^*\| + tE.$$

(b). Show

$$\|\mathbf{x}_{k+1} - \mathbf{x}^*\| \leq q\|\mathbf{x}_k - \mathbf{x}^*\| + tE,$$

where $q = \sqrt{1 - mt}$. Hint: Use the result in our analysis for the noiseless case to bound $\|\tilde{\mathbf{x}}_{k+1} - \mathbf{x}^*\|$.

(c). Show by induction that

$$\|\mathbf{x}_k - \mathbf{x}^*\| \leq q^k \|\mathbf{x}_0 - \mathbf{x}^*\|_2 + \frac{1 - q^k}{1 - q} tE$$

(d). Conclude

$$\limsup_{k \rightarrow \infty} \|\mathbf{x}_k - \mathbf{x}^*\| \leq \frac{tE}{1 - q} \leq \frac{2E}{m}$$

i.e. $\mathbf{x}_k$ “converges” to a ball of radius $\frac{2E}{m}$ centered at $\mathbf{x}^*$.

Proof. (a). Note $\mathbf{x}_{k+1} = \tilde{\mathbf{x}}_{k+1} - t\boldsymbol{\varepsilon}_k$. By the triangle inequality,

$$\begin{aligned} \|\mathbf{x}_{k+1} - \mathbf{x}^*\| &= \|\tilde{\mathbf{x}}_{k+1} - \mathbf{x}^* - t\boldsymbol{\varepsilon}_k\| \\ &\leq \|\tilde{\mathbf{x}}_{k+1} - \mathbf{x}^*\| + \|t\boldsymbol{\varepsilon}_k\| \quad \text{by triangle inequality} \\ &= \|\tilde{\mathbf{x}}_{k+1} - \mathbf{x}^*\| + t\|\boldsymbol{\varepsilon}_k\| \\ &\leq \|\tilde{\mathbf{x}}_{k+1} - \mathbf{x}^*\| + tE \end{aligned}$$

(b). By our analysis of the noiseless case,

$$\|\tilde{\mathbf{x}}_{k+1} - \mathbf{x}^*\|^2 \leq (1 - mt)\|\mathbf{x}_k - \mathbf{x}^*\|^2 \implies \|\tilde{\mathbf{x}}_{k+1} - \mathbf{x}^*\| \leq q\|\mathbf{x}_k - \mathbf{x}^*\|$$

Plugging into (a),

$$\|\mathbf{x}_{k+1} - \mathbf{x}^*\| \leq q\|\mathbf{x}_k - \mathbf{x}^*\| + tE,$$

(c). The base case $k = 0$ is trivially true. Assume the inequality holds for k . For $k + 1$,

$$\begin{aligned} \|\mathbf{x}_{k+1} - \mathbf{x}^*\| &\leq q\|\mathbf{x}_k - \mathbf{x}^*\| + tE \\ &\leq q \left[q^k \|\mathbf{x}_0 - \mathbf{x}^*\| + \frac{1 - q^k}{1 - q} tE \right] + tE \\ &= q^{k+1} \|\mathbf{x}_0 - \mathbf{x}^*\| + \frac{1 - q^{k+1}}{1 - q} tE \end{aligned}$$

which completes the induction.

(d).

$$\limsup_{k \rightarrow \infty} \|\mathbf{x}_k - \mathbf{x}^*\| \leq \frac{tE}{1 - q} = \frac{tE(1 + q)}{1 - q^2} = \frac{E(1 + q)}{m} \leq \frac{2E}{m}$$

□